🎵 Intelligent Music Discovery & Recommendation System

An end-to-end Machine Learning and NLP-powered music recommendation platform that classifies, clusters, and matches songs using semantic lyric understanding and metadata-driven search. The system combines traditional fuzzy matching with transformer-based embeddings to deliver highly relevant music recommendations through an interactive Streamlit web application.

🚀 Features

🔍 Dual-Engine Search System

1. Fuzzy Metadata Search

- High-speed text matching using `RapidFuzz`
- Searches across:
- Song titles
- Artists
- Categories
- Clusters
- Optimized weighted scoring for highly relevant recommendations

2. NLP Semantic Search

- Uses `SentenceTransformers (all-MiniLM-L6-v2)` for contextual lyric understanding
- Encodes user queries into dense vector embeddings
- Computes cosine similarity against precomputed song embedding matrices
- Enables mood-based and semantic music discovery

🎼 Song Classification Interface

Users can submit:

- Lyrics
- Song descriptions
- Titles
- Mood-based prompts

The system dynamically preprocesses and embeds textual inputs to perform real-time category prediction and recommendation generation.

⚡ Performance Optimizations

- Streamlit caching using:
- `@st.cache_resource`
- `@st.cache_data`
- Optimized vector similarity operations using NumPy matrix computations
- Precomputed embedding matrices for low-latency semantic retrieval
- Fast multi-candidate ranking pipeline

🎨 User Interface

The application features a modern Spotify-inspired interface including:

- Responsive layouts
- Custom CSS styling
- Animated glowing radial backgrounds
- Interactive result cards
- Inline recommendation metrics
- Direct YouTube integration links

📂 Project Structure

```
├── music recommendation system2.ipynb   # Data preprocessing & model training pipeline
├── streamlit_app.py                     # Interactive Streamlit application
├── best_model.pkl                       # Serialized classification model
├── embedding_model.pkl                  # SentenceTransformer embedding model
└── songs_clustered.csv                  # Processed music dataset
```

⚙️ Machine Learning Pipeline

🧹 Data Preprocessing

The training pipeline performs:

- Regex-based lyric cleaning
- Tokenization

- Stopword removal using NLTK
- Lemmatization using WordNetLemmatizer
- Text normalization and formatting

Primary dataset:

- `spotify_millsongdata.csv`

🧠 Embedding Generation

Lyrics are converted into dense semantic vectors using:

```
SentenceTransformer("all-MiniLM-L6-v2")
```

These embeddings enable contextual similarity matching between songs and user queries.

📊 Model Training

The notebook benchmarks multiple machine learning classifiers and exports the best-performing model using `joblib`.

Tasks include:

- Feature extraction
- Classification training
- Validation benchmarking
- Inference optimization

🛠️ Installation & Setup

Prerequisites

- Python 3.9+
- pip package manager

1️⃣ Clone Repository

```
git clone https://github.com/your-username/music-recommendation-system.git
cd music-recommendation-system
```

2️⃣ Install Dependencies

```
pip install streamlit pandas numpy joblib scikit-learn sentence-transformers
rapidfuzz nltk
```

3️⃣ Download NLP Resources

```
python -c "import nltk; nltk.download('punkt'); nltk.download('stopwords');
nltk.download('wordnet')"
```

▶️ Running the Application

Ensure the following files exist in the root directory:

- `songs_clustered.csv`
- `best_model.pkl`
- `embedding_model.pkl`

Run the application:

```
streamlit run streamlit_app.py
```

💡 Recommendation Logic

🔎 Weighted Fuzzy Matching

The metadata search engine prioritizes important fields using weighted scoring:

$$\text{Score} = \max\left\{\text{FuzzRatio}(\text{Query}, \text{Song}) \times 1.40 \; \text{FuzzRatio}(\text{Query}, \text{Artist}) \times 1.25 \; \text{FuzzRatio}(\text{Query}, \text{Label}) \times 1.10 \; \text{F}\right.$$

🧠 Semantic NLP Ranking

Semantic search combines cosine similarity with fuzzy metadata boosting:

$$\text{Rank} = (\text{Cosine Similarity} \times 60) + (\text{Fuzzy Metadata Boost} \times 0.40)$$

This hybrid strategy balances:

- Deep contextual understanding
- Structural keyword relevance
- Metadata precision

🧰 Technologies Used

Languages

- Python

Machine Learning & NLP

- Scikit-learn
- SentenceTransformers
- NLTK

Data Processing

- Pandas
- NumPy

Search & Ranking

- RapidFuzz

Frontend & Deployment

- Streamlit
- Custom CSS

📈 Key Highlights

- Semantic recommendation engine for 57K+ tracks
- Hybrid NLP + fuzzy search architecture
- Real-time recommendation and classification workflows
- Optimized vector similarity search pipelines
- Interactive deployment-ready web application

📜 License

This project is intended for educational and research purposes.